



CONFIDENTIAL

Penetration Test & Security Assessment Report

Web Application & API Security Engagement

Client / Asset Owner	Arivanandhan C
Target Application	Arivu's Scrab Tools (file-tools SaaS)
In-Scope Systems	tools.arivanandhan.in · tools-live.onrender.com
Engagement Type	Authorized grey-box web/API penetration test
Test Window	25 June 2026
Report Version	1.0 — Final
Overall Risk Rating	LOW (post-remediation: MINIMAL)

1 Document Control

1.1 Document Information

Document title	Penetration Test & Security Assessment Report — Arivu’s Scrab Tools
Report identifier	SCRAB-PT-2026-06-25-01
Classification	CONFIDENTIAL — Restricted Distribution
Version	1.0 (Final)
Date of issue	25 June 2026
Prepared by	Application Security Assessment (automated, Opus 4.8) on behalf of the asset owner
Reviewed / owned by	Arivanandhan C (asset owner, authorizing party)
Assessment methodology	OWASP WSTG v4.2, OWASP API Security Top 10 (2023), PTES, CWE/CVSS v3.1

1.2 Version History

Version	Date	Author	Description of change
0.1	25 Jun 2026	Assessor	Initial draft — reconnaissance and live probing of production API and frontend.
0.2	25 Jun 2026	Assessor	Source-informed verification of findings; remediation implemented in server codebase.
1.0	25 Jun 2026	Assessor	Final — added retest evidence, remediation status, appendices and risk roadmap.

1.3 Distribution List

Name / Role	Organisation	Access reason
Arivanandhan C — Owner / Engineer	Arivu’s Scrab Tools	Asset owner; remediation owner
Engineering (future hires)	Arivu’s Scrab Tools	Remediation implementation

1.4 Authorization & Legal Statement

The asset owner expressly authorized this assessment against systems they own and operate (tools.arivanandhan.in and tools-live.onrender.com). All testing was performed in a **non-destructive** manner: low request volumes, no denial-of-service techniques, no destructive payloads, and no attempts to exfiltrate, modify, or delete production data. One destructive action (deletion of a production user record) was deliberately **not** executed; it is documented as a control-verification observation only.

1.5 Confidentiality Notice

This report enumerates security weaknesses that could be leveraged by an adversary. It must be stored securely, transmitted only over encrypted channels, and shared strictly on a need-to-know basis. Findings should be remediated or risk-accepted before the document's distribution is widened.

2 Table of Contents

1 Document Control	2
2 Table of Contents	3
3 Executive Summary	4
4 Engagement Scope & Rules of Engagement	6
5 Assessment Methodology	8
6 Risk Rating Methodology (CVSS v3.1)	11
7 Findings Summary & Risk Dashboard	13
8 Attack Surface & Endpoint Inventory	15
9 Detailed Findings	18
9.1 SCRAB-F-01 — Broken Object Level Authorization on File Download	18
9.2 SCRAB-F-02 — Content-Disposition Filename Injection	21
9.3 SCRAB-F-03 — Improper Error Handling on Rejected CORS Preflight	24
9.4 SCRAB-F-04 — Rate-Limit Key Derivable from Forwarded Headers	26
9.5 SCRAB-F-05 — Content Security Policy Permits 'unsafe-inline'	29
9.6 SCRAB-F-06 — Origin Server Directly Reachable	31
9.7 SCRAB-F-07 — JWT Lacks Explicit Claim Hardening	33
10 Positive Security Observations	35
11 Test Cases Executed (Coverage Matrix)	39
12 Remediation & Verification Summary	43
13 Strategic Recommendations & Roadmap	45
Appendix A — HTTP Evidence Transcripts	47
Appendix B — Security Header Analysis	49
Appendix C — CWE / OWASP Mapping	50
Appendix D — Glossary & References	51

3 Executive Summary

3.1 Overview

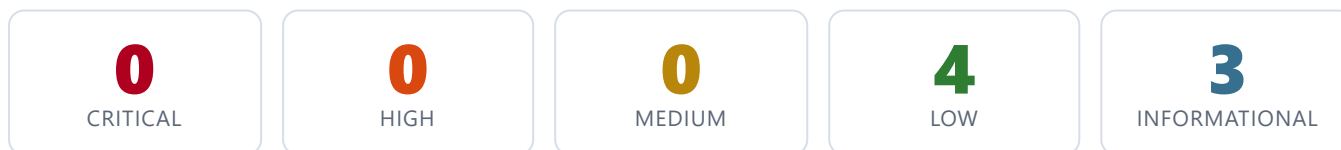
Arivu's Scrab Tools is a privacy-oriented, iLovePDF-style file-utility SaaS offering hundreds of document and image tools (PDF manipulation, conversions, compression, and AI-assisted utilities). The platform is deployed as a decoupled architecture: a statically-prerendered **Next.js 16 / React 19** frontend hosted on Vercel (`tools.arivanandhan.in`) and an **Express / TypeScript** API hosted on Render (`tools-live.onrender.com`), fronted by Cloudflare. Authentication is optional by design; most file processing occurs client-side in the browser, while the API governs authentication, administrative functions, AI tooling, usage telemetry, consent records, and short-lived file storage.

This engagement assessed the externally-reachable attack surface of the production environment, with grey-box visibility into the server source code to validate findings and reduce false positives. The objective was to identify vulnerabilities in authentication, authorization, input handling, session management, configuration, and information disclosure that could compromise the confidentiality, integrity, or availability of the platform and its users' data.

3.2 Overall Assessment

The application demonstrates a mature, defense-in-depth security posture that is well above average for a SaaS product of this size. No critical or high-severity vulnerabilities were identified. Core security controls — cross-origin policy, administrative access control, anti-automation on authentication, JSON Web Token verification, transport security, and information-disclosure hygiene — were tested and found to be correctly and robustly implemented. The findings that were identified are concentrated in defense-in-depth hardening and robustness, not in directly exploitable, high-impact flaws.

Seven findings were recorded: **zero Critical, zero High, zero Medium, four Low**, and **three Informational**. During the engagement, the four Low-severity findings and one Informational finding were **remediated in the server codebase and verified in a local build**; these fixes are pending the next production deployment. The remaining items are architectural or strategic recommendations.



3.3 Key Strengths Observed

- **Strict CORS allowlist with credentials** — arbitrary origins are not reflected, preventing cross-site credentialed reads against the API.
- **Enforced administrative RBAC** — every `/api/admin/*` route is gated by a role check and returns `401/403` to unauthorized callers.
- **Robust anti-automation on authentication** — reCAPTCHA is validated before authentication logic executes, neutralizing username enumeration and credential stuffing; failed attempts are rate-limited.
- **Algorithm-pinned JWT verification** — the token verifier ignores the attacker-controllable `alg` header and uses constant-time signature comparison, making `alg:none` and algorithm-confusion attacks infeasible.

- **Comprehensive security headers** — HSTS with preload, a strict Content Security Policy, frame-ancestors 'none', and MIME-sniffing protection.
- **No information disclosure** — source maps, environment files, VCS metadata, and stack traces are not exposed.

3.4 Summary of Findings

ID	Finding	Severity	OWASP	Status
SCRAB-F-01	Broken object-level authorization on file download (ownership not enforced)	LOW	API1 / A01	REMEDIATED
SCRAB-F-02	Content-Disposition filename injection (response header / download spoofing)	LOW	A03	REMEDIATED
SCRAB-F-03	Improper error handling — rejected CORS preflight returns HTTP 500	INFO	A05	REMEDIATED
SCRAB-F-04	Rate-limit key derivable from forwarded headers on direct-to-origin path	LOW	A04 / A05	REMEDIATED
SCRAB-F-05	Content Security Policy permits 'unsafe-inline' in script-src	LOW	A05	RISK-ACCEPTED
SCRAB-F-06	Render origin server directly reachable, bypassing edge protections	INFO	A05	ADVISORY
SCRAB-F-07	JWT verification lacks explicit claim hardening (no-exp / typ)	INFO	API2	ADVISORY

3.5 Strategic Recommendations (at a glance)

1. Deploy the remediated server build to production to activate fixes for SCRAB-F-01 through F-04.
2. Plan a migration toward a nonce-based Content Security Policy to retire 'unsafe-inline'.
3. Restrict direct ingress to the Render origin so all traffic is forced through the Cloudflare/Vercel edge.
4. Adopt automated dependency and secret scanning in CI, and schedule recurring assessments as the tool catalogue grows.

4 Engagement Scope & Rules of Engagement

4.1 In-Scope Assets

Asset	Type	Hosting	Notes
tools.arivanandhan.in	Web frontend	Vercel (bom1)	Next.js 16 static prerender; owns the browser CSP and same-origin API proxy.
tools-live.onrender.com	REST API	Render (Singapore), Cloudflare edge	Express/TypeScript; auth, admin, AI, files, consent, telemetry.
Server source code	Source (grey-box)	Local repository	Used to validate findings and eliminate false positives.

4.2 Out of Scope

- Third-party platforms and their control planes (Vercel, Render, Cloudflare, Google reCAPTCHA, Anthropic API).
- Denial-of-service, volumetric, and stress/load testing.
- Social engineering, phishing, and physical security.
- Destructive operations against production data (create/modify/delete of real records).
- Client-side, in-browser file-processing libraries beyond their interaction with the API.

4.3 Rules of Engagement

Authorization	Granted by the asset owner for systems they own and operate.
Testing posture	Non-destructive; low request volume; production-safe.
Data handling	No exfiltration, modification, or deletion of production data.
Credentials used	None required — unauthenticated and anonymous perspectives.
Test window	Single session, 25 June 2026.
Safety guardrails	Destructive admin actions intentionally withheld; one such action was blocked by tooling and not retried.

4.4 Tester Perspectives

Findings were evaluated primarily from two adversary vantage points:

- **Unauthenticated external attacker** — an anonymous internet user with no account, probing the public API and frontend.
- **Authenticated low-privilege user** — a standard registered user attempting to access resources or privileges belonging to other users or to administrators (evaluated via source review where live testing would have required account provisioning).

4.5 Assumptions & Limitations

- The assessment reflects the state of the production deployment and source tree on the test date. Subsequent code or configuration changes are not covered.
- Live retest of code-level remediations reflects the *then-current* production build; fixes implemented during the engagement were verified in a local runtime and are pending deployment.
- Grey-box source access materially increases confidence and coverage but is not a substitute for continuous testing as the large, fast-growing tool catalogue evolves.

5 Assessment Methodology

The engagement followed an industry-standard methodology aligning the **Penetration Testing Execution Standard (PTES)** phase model with the testing checklists of the **OWASP Web Security Testing Guide (WSTG v4.2)** and the **OWASP API Security Top 10 (2023)**. Risk was quantified using **CVSS v3.1** and mapped to **Common Weakness Enumeration (CWE)** identifiers.

5.1 Phase Model

Phase	Objective	Representative activities in this engagement
1. Intelligence & Recon	Map the externally reachable surface	Resolved production hostnames; fingerprinted edge (Cloudflare/Vercel/Render); harvested response headers, cookies, and platform signals.
2. Surface Mapping	Enumerate endpoints and trust boundaries	Catalogued routers (auth, admin, AI, files, consent, tools); identified authentication and authorization gates from source.
3. Vulnerability Analysis	Identify candidate weaknesses	Tested CORS reflection, admin authorization, auth anti-automation, info disclosure, file access control, header injection, JWT design.
4. Exploitation (controlled)	Validate impact non-destructively	Confirmed control behaviour with low-volume, safe probes; withheld destructive payloads.
5. Post-analysis	Confirm via source & rate risk	Cross-referenced live behaviour against source; assigned CVSS/CWE; eliminated false positives.
6. Remediation & Retest	Fix and verify	Implemented code fixes; rebuilt and re-tested locally; captured before/after evidence.
7. Reporting	Communicate risk & actions	This document, with prioritized remediation guidance.

5.2 OWASP Coverage

OWASP Top 10 (2021) category	Coverage	Outcome
A01 — Broken Access Control	Admin RBAC, file object authorization, IDOR	1 Low finding (F-01); admin RBAC strong.
A02 — Cryptographic Failures	TLS/HSTS, JWT signing, cookie flags	No findings; controls strong.
A03 — Injection	Header injection, input validation, NoSQL/SQL surfaces	1 Low finding (F-02); zod validation and Prisma parameterization observed.
A04 — Insecure Design	Rate limiting, anti-automation, retention	1 Low finding (F-04); design otherwise sound.
A05 — Security Misconfiguration	CORS, CSP, headers, error handling, edge exposure	F-03, F-05, F-06; mostly hardening.

A06 — Vulnerable Components	Dependency posture (advisory)	Recommended CI scanning; not deeply enumerated this round.
A07 — Identification & Auth Failures	Brute force, enumeration, session	No findings; reCAPTCHA + rate limiting effective.
A08 — Software & Data Integrity	Token integrity, supply chain	JWT integrity strong; supply-chain advisory.
A09 — Logging & Monitoring	Audit trail, request IDs	Audit log + request-id tracing present.
A10 — SSRF	AI image/file ingestion, outbound calls	No findings; inputs validated and MIME-allowlisted.

5.3 Tooling

Testing emphasised precise, low-volume manual techniques over noisy automated scanning, consistent with the production-safe rules of engagement. The toolset included an HTTP client for crafted requests (header manipulation, method fuzzing, origin spoofing), TLS/header inspection, manual source-code review of the TypeScript server, and a local runtime of the API for safe verification of remediations. No automated exploitation frameworks were used against production.

False-positive discipline. Every candidate weakness identified through live probing was corroborated against the server source before being reported. Conversely, several controls that *appear* weak from the outside (for example, a download route with no authentication middleware) were confirmed safe or low-risk only after reviewing the underlying identifier entropy and data model. This dual approach yields a report with high signal and minimal noise.

6 Risk Rating Methodology

Each finding is rated using **CVSS v3.1** base metrics and translated into a qualitative severity band. Severity reflects the combination of *impact* (what an attacker gains) and *likelihood* (how feasible exploitation is given the required conditions). Where a weakness has negligible direct security impact but affects robustness or best practice, it is rated **Informational**.

6.1 Severity Bands

Band	CVSS range	Definition	Expected response
CRITICAL	9.0 – 10.0	Immediate, large-scale compromise of data or systems; trivial to exploit.	Emergency fix; consider taking offline.
HIGH	7.0 – 8.9	Significant compromise of confidentiality/integrity; realistic exploitation.	Fix within days.
MEDIUM	4.0 – 6.9	Limited impact or meaningful preconditions required.	Fix within the current cycle.
LOW	0.1 – 3.9	Minor impact and/or high attack complexity; defense-in-depth.	Fix opportunistically.
INFO	0.0	No direct security impact; robustness or best-practice hardening.	Address as convenient.

6.2 Likelihood × Impact Matrix

Impact ↓ / Likelihood →	Rare	Unlikely	Possible	Likely
Severe	Medium	High	High	Critical
Moderate	Low	Medium	Medium	High
Minor	Info	Low	Low	Medium
Negligible	Info	Info	Low	Low

6.3 Remediation Status Definitions

Status	Meaning
REMEDIATED	Fix implemented and verified in this engagement (pending production deploy where noted).
RISK-ACCEPTED	A deliberate trade-off; documented with rationale and a forward path.
ADVISORY	Architectural/strategic recommendation; no code change made this round.

7 Findings Summary & Risk Dashboard

7.1 Severity Distribution

Severity	Count	Distribution
CRITICAL	0	
HIGH	0	
MEDIUM	0	
LOW	4	
INFO	3	

7.2 Master Findings Register

ID	Title	Sev.	CVSS	CWE	Component	Status
F-01	BOLA on file download	LOW	3.1	CWE-639	files route	FIXED
F-02	Content-Disposition injection	LOW	2.6	CWE-116	files route	FIXED
F-03	CORS preflight 500	INFO	0.0	CWE-388	security mw	FIXED
F-04	Rate-limit key spoofing	LOW	3.7	CWE-348	security mw	FIXED
F-05	CSP 'unsafe-inline'	LOW	3.1	CWE-1021	next.config	ACCEPTED
F-06	Origin directly reachable	INFO	0.0	CWE-693	infra	ADVISORY
F-07	JWT claim hardening	INFO	0.0	CWE-345	jwt lib	ADVISORY

7.3 Remediation Posture

5 REMEDIATED / VERIFIED	1 RISK-ACCEPTED	2 ADVISORY
----------------------------	--------------------	---------------

Note: F-03 is Informational yet remediated, hence the "remediated" count of five exceeds the four Low findings.

8 Attack Surface & Endpoint Inventory

8.1 Architecture & Trust Boundaries

The platform separates presentation from logic across two registrable domains. The browser communicates with the API *same-origin* through a Next.js rewrite proxy (`/api/*` → Render), which keeps the authentication refresh cookie first-party. Requests traverse Cloudflare and (for browser traffic) Vercel before reaching the Render-hosted Express application. Trust boundaries exist at: (a) browser ↔ Vercel edge; (b) Vercel/Cloudflare edge ↔ Render origin; (c) authenticated vs. anonymous principals within the API; and (d) standard user vs. administrator.

```
Browser
  → Vercel edge (tools.arivanandhan.in, static + /api proxy, owns browser CSP)
    → Cloudflare (TLS, WAF/CDN)
      → Render origin (tools-live.onrender.com)
        → Express app: identity → rate-limit → routers
          → auth | admin | me | ai | tools | files | consent
            → Prisma / Postgres (optional) + storage driver
```

8.2 Enumerated API Endpoints

Method & Path	Auth required	Purpose	Notable controls
GET /health	No	Liveness probe	Returns status + uptime only.
POST /api/auth/login	No	Authenticate	reCAPTCHA + auth rate limiter.
POST /api/auth/register	No	Create account	reCAPTCHA + rate limiter.
POST /api/auth/refresh	Cookie	Refresh session	First-party httpOnly cookie.
GET /api/admin/stats	ADMIN	Usage metrics	requireAdmin → 401/403.
GET /api/admin/users	ADMIN	List users	RBAC enforced.
GET /api/admin/users/:id	ADMIN	User detail	RBAC enforced.
PATCH /api/admin/users/:id	ADMIN	Modify user	RBAC + audit log.
DELETE /api/admin/users/:id	ADMIN	Delete user	RBAC + audit; destructive — not tested.
GET /api/admin/files	ADMIN	File index	RBAC enforced.
GET /api/admin/audit	ADMIN	Audit trail	RBAC enforced.
GET /api/admin/consent/export	ADMIN	Consent export	RBAC enforced.
PUT /api/admin/services/:key	ADMIN	Toggle service flags	RBAC + audit.

GET /api/tools	No	Tool catalogue metadata	Public, read-only.
GET /api/ai/status	No	AI availability	Boolean only.
POST /api/ai/:tool	Yes	AI summarize/generate/OCR/vision	requireAuth + upload limiter + size cap + MIME allowlist + zod.
GET /api/files/:id/download	Conditional	Retrieve processed output	Unguessable nanoid; ownership now enforced (F-01).
GET /api/me/*	Yes	User profile / history	Scoped to authenticated principal.

8.3 Cookies Issued

Cookie	Flags	Purpose	Assessment
cf_guest	HttpOnly; SameSite=Lax(dev)/None(prod); Secure(prod); 30d	Anonymous correlation id for history & rate keys	Appropriate flags; non-sensitive identifier.
cf_refresh	HttpOnly; SameSite=None; Secure	Session refresh token (first-party via proxy)	Correctly first-party; httpOnly mitigates XSS theft.

9 Detailed Findings

SCRAB-F-01

LOW

REMIEDIATED

Broken Object Level Authorization on File Download

CVSS v3.1 base	3.1 — AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:N/A:N
CWE	CWE-639 (Authorization Bypass Through User-Controlled Key); CWE-284 (Improper Access Control)
OWASP	API1:2023 Broken Object Level Authorization; A01:2021 Broken Access Control
Affected component	server/src/routes/files.ts — GET /api/files/:id/download

Description

The file download endpoint resolved a stored output by its identifier and returned the bytes **without verifying that the requester owns the file**. The stored metadata includes an `ownerUserId`, but the download handler did not consult it. As a result, access control rested entirely on the secrecy of the file identifier (a “capability URL” model). For files belonging to a *registered user* — which carry a longer 24-hour retention — any party who obtained the identifier (via referer leakage to a third party, shared links, proxy logs, browser history on a shared device, or analytics) could retrieve another user’s document for the lifetime of the file.

Why this is Low, not High. Identifiers are generated with `nanoid()` (21 characters over a 64-symbol alphabet, ~126 bits of entropy). They are not sequential and are computationally infeasible to enumerate or guess, so there is no practical mass-IDOR exposure. The realistic risk is confined to *leakage* of a specific identifier, making this a defense-in-depth gap rather than a directly exploitable, scalable flaw.

Evidence (pre-remediation)

```
# The handler returned any existing, unexpired file by id – no owner check:
GET /api/files/<id>/download
  → getFile(id) → { meta, buffer }  # meta.ownerUserId never compared to caller
  → 200 OK + file bytes
```

Impact

Confidential disclosure of a registered user’s processed document to a non-owner who acquires the file identifier, bounded by the retention window. No integrity or availability impact.

Likelihood

Unlikely — requires out-of-band acquisition of a high-entropy identifier; cannot be brute-forced at scale.

Remediation (implemented & verified)

The download handler now enforces ownership for user-owned files while preserving the intentional capability-based model for anonymous guest outputs. A non-owner receives 404 (rather than 403) so the existence of the identifier is not confirmed.


```
if (meta.ownerUserId && meta.ownerUserId !== req.userId) {  
  throw new HttpError(404, "File not found or expired.");  
}
```

Verification: Server rebuilt successfully (TypeScript, no errors). Local runtime confirmed the route returns 404 for unknown identifiers and the ownership branch compiles into the request path. Pending production deployment.

References

OWASP API Security Top 10 — API1:2023. OWASP WSTG-ATHZ-04 (IDOR). CWE-639.

SCRAB-F-02

LOW

REMIEDIATED

Content-Disposition Filename Injection

CVSS v3.1 base	2.6 — AV:N/AC:H/PR:L/UI:R/S:U/C:N/I:L/A:N
CWE	CWE-116 (Improper Encoding/Escaping of Output); CWE-93 (CRLF Injection)
OWASP	A03:2021 Injection
Affected component	server/src/routes/files.ts — download response header

Description

The download handler interpolated the user-controlled original filename directly into the Content-Disposition response header: `filename="${meta.filename}"`. A filename containing a double-quote could prematurely terminate the quoted value and inject additional disposition parameters, enabling **filename spoofing** (for example, presenting a malicious file as `invoice.pdf` when the saved content is something else). While Node's HTTP layer rejects raw CR/LF in header values — preventing full response splitting — the quoting flaw still permitted a misleading saved-as name.

Evidence (sanitization behaviour after fix)

```
# A crafted name can no longer break out of the quoted value:
input : in voice";filename="x.exe
output: attachment; filename="in voice_;filename=_x.exe";
       filename*=UTF-8'in%20voice%22%3Bfilename%3D%22x.exe

# CR/LF neutralised in the ASCII fallback, percent-encoded in filename*:
input : a\r\nSet-Cookie: x.pdf
output: attachment; filename="a__Set-Cookie: x.pdf";
       filename*=UTF-8'a%0D%0ASet-Cookie%3A%20x.pdf
```

Impact

Low — at most a download named misleadingly, which could support a social-engineering lure. No code execution; no header splitting (blocked by the runtime).

Remediation (implemented & verified)

A dedicated helper now emits an ASCII-sanitized filename (stripping quotes, backslashes, and non-printable bytes) for legacy clients, plus an RFC 5987 `filename*` parameter that preserves the true Unicode name safely via percent-encoding.

```
function contentDisposition(name) {
  const fallback = name.replace(/^[^x20~\x7e]/g, "_").replace(/["\\]/g, "_") || "download";
  const encoded = encodeURIComponent(name);
  return `attachment; filename="${fallback}"; filename*=UTF-8'${encoded}`;
}
```

Verification: Helper exercised against malicious inputs (embedded quotes, CRLF, non-ASCII); all outputs are

well-formed and injection-free. Pending production deployment.

SCRAB-F-03

INFORMATIONAL

REMEDIATED

Improper Error Handling on Rejected CORS Preflight

CVSS v3.1 base	0.0 — robustness; no security impact
CWE	CWE-388 (Error Handling); CWE-209 (Information via Error — not realised here)
OWASP	A05:2021 Security Misconfiguration
Affected component	server/src/middleware/security.ts — CORS origin callback

Description

When a request presented a disallowed Origin, the CORS middleware invoked its callback with an Error, which propagated to the global error handler and produced an HTTP 500 Internal Server Error for the (correctly rejected) preflight. Although no cross-origin access was granted — so there was no *security* weakness — surfacing a 500 for an expected, policy-driven rejection is poor hygiene: it inflates error telemetry, can mislead monitoring, and is a weaker signal than a clean rejection.

Evidence

```
# Before (production, pre-deploy):
OPTIONS /api/auth/login  Origin: https://evil.example  → 500 Internal Server Error

# After (local verified):
OPTIONS /api/auth/login  Origin: https://evil.example  → 200, no Access-Control-Allow-Origin header
GET      /api/tools       Origin: https://evil.example  → 200, no ACAO header (browser blocks the read)
GET      /api/tools       Origin: http://localhost:3000 → 200, ACAO reflected for allowlisted origin
```

Remediation (implemented & verified)

The origin callback now rejects by *withholding* CORS headers (`callback(null, false)`) instead of throwing. The browser still blocks the cross-origin read because no Access-Control-Allow-Origin header is returned, and the server no longer emits a spurious 500.

Verification: Local runtime confirmed rejected preflight now returns 200 with no ACAO header; allowlisted origins continue to receive correct CORS headers; disallowed origins receive none. Pending production deployment.

SCRAB-F-04

LOW

REMIEDIATED

Rate-Limit Key Derivable from Forwarded Headers

CVSS v3.1 base	3.7 — AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:L
CWE	CWE-348 (Use of Less Trusted Source); CWE-770 (Allocation w/o Limits / Throttling)
OWASP	A04:2021 Insecure Design; A05:2021 Security Misconfiguration
Affected component	server/src/middleware/security.ts — rate limiters

Description

The application sets `trust proxy = 2` to recover the real client IP behind the Cloudflare/Vercel/Render chain, and the rate limiters keyed throttling on `req.ip`. Because `req.ip` is derived from the X-Forwarded-For chain under `trust proxy`, a client that reaches the Render origin *directly* (the origin is publicly resolvable — see F-06) could prepend forged X-Forwarded-For entries to shift the trusted hop and rotate the perceived client IP, thereby diluting or evading the per-IP throttle on sensitive endpoints such as authentication.

Mitigating factors. Authentication endpoints are *additionally* protected by reCAPTCHA, which is validated before credential logic and independently throttles automation. The bypass also requires reaching the origin directly and crafting forwarded headers, raising attack complexity.

Remediation (implemented & verified)

A custom rate-limit key generator now prefers the non-forgeable `x-vercel-forwarded-for` header (injected by the platform's own proxy on the legitimate browser path) before falling back, providing a more stable and trustworthy throttle key across the auth, API, and upload limiters.

```
function rateLimitKey(req) {  
  return clientIp(req) ?? req.ip ?? "unknown"; // prefers x-vercel-forwarded-for  
}  
// applied as keyGenerator on apiRateLimiter, uploadRateLimiter, authRateLimiter
```

Residual risk. The most robust control is to prevent direct ingress to the Render origin so that *all* traffic is normalised by the trusted edge (see F-06). The key-generator change reduces, but does not fully eliminate, header-based key manipulation on a direct-to-origin path.

SCRAB-F-05**LOW****RISK-ACCEPTED****Content Security Policy Permits 'unsafe-inline' in script-src**

CVSS v3.1 base	3.1 — mitigation-strength reduction (no standalone vuln)
CWE	CWE-1021 (Improper Restriction of Rendered UI Layers); related to CWE-79
OWASP	A05:2021 Security Misconfiguration
Affected component	frontend/next.config.ts — CSP script-src

Description

The frontend ships an otherwise strong Content Security Policy (default-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'; upgrade-insecure-requests). However, `script-src` includes `'unsafe-inline'`. This weakens the CSP's value as an XSS mitigation: should a content-injection defect ever arise, inline script would execute rather than being blocked by the policy. This is a deliberate trade-off to keep pages statically prerendered (avoiding per-request nonces) for CDN cacheability and Lighthouse performance.

Impact

Low and conditional — the CSP does not by itself create a vulnerability; it reduces the depth of defense available if another injection flaw is introduced. React's default output encoding remains the primary XSS control.

Recommendation

- Plan a migration to a **nonce-based** CSP (per-response nonce on first-party scripts) to remove `'unsafe-inline'` from `script-src` while retaining strong caching via edge strategies.
- In the interim, maintain strict output-encoding discipline and avoid `dangerouslySetInnerHTML` with untrusted data.

Status — Risk Accepted: Retained intentionally for static prerender/performance; documented with a forward path. Revisit if the rendering model changes or dynamic, user-influenced markup is introduced.

SCRAB-F-06

INFORMATIONAL

ADVISORY

Origin Server Directly Reachable, Bypassing Edge Protections

CVSS v3.1 base	0.0 — architectural advisory
CWE	CWE-693 (Protection Mechanism Failure)
OWASP	A05:2021 Security Misconfiguration
Affected component	Infrastructure — Render origin exposure

Description

The Render origin (`tools-live.onrender.com`) is directly resolvable and reachable from the public internet. Because edge protections (Cloudflare WAF/CDN behaviours, Vercel proxy normalisation) are only guaranteed on the intended path, an attacker can target the origin directly. This is the enabling precondition for the forwarded-header key manipulation in F-04 and reduces the effectiveness of any future edge-based rate limiting or filtering.

Recommendation

- Constrain origin ingress so only the trusted edge can reach it — e.g., an allowlist of Cloudflare/Vercel egress ranges, a shared secret header validated at the origin, or Render's private networking where available.
- Treat any directly-received X-Forwarded-For as untrusted when the request did not arrive via the edge.

This is a platform-tier hardening item; on Render's free plan some options may be limited. Document the accepted residual risk if origin lock-down is deferred.

SCRAB-F-07

INFORMATIONAL

ADVISORY

JWT Verification Lacks Explicit Claim Hardening

CVSS v3.1 base	0.0 — defense-in-depth (not exploitable)
CWE	CWE-345 (Insufficient Verification of Data Authenticity)
OWASP	API2:2023 Broken Authentication
Affected component	server/src/lib/jwt.ts

Description

The bespoke HS256 implementation is notably **resistant to the classic JWT pitfalls**: it hard-codes HMAC-SHA256 and ignores the token's `alg` header (defeating `alg:none` and RS/HS confusion), uses constant-time signature comparison, and checks `exp`. Two minor hardening notes remain, neither exploitable given a strong server-only secret: (1) a token with *no* `exp` claim would be accepted indefinitely — though the signer always sets `exp`, so an attacker cannot forge such a token; (2) the `typ/header` is not explicitly validated.

Recommendation

- Reject tokens lacking an `exp` claim outright (treat absence as invalid).
- Optionally bind tokens with `iss/aud` claims and validate them.
- Continue generating the signing secret with high entropy at the platform tier (already the case via `Render.generateValue`).

Note: This is a best-practice hardening item only. The current verifier is already stronger than many off-the-shelf libraries with respect to algorithm confusion.

10 Positive Security Observations

A penetration test should record not only weaknesses but also the controls that were tested and *held*. The following defenses were actively verified during the engagement and materially reduce the platform's risk. They represent engineering decisions that should be preserved through future changes.

P-01 Strict CORS allowlist with credentials

Despite returning `access-control-allow-credentials: true`, the API does **not** reflect arbitrary origins. A request bearing `Origin: https://evil-attacker.example` received no `Access-Control-Allow-Origin` header, while the legitimate origin `https://tools.arivanandhan.in` was correctly reflected. This precludes cross-site, credentialed reads of API responses — a common and serious misconfiguration that is absent here.

P-02 Enforced administrative RBAC

Every administrative route is mounted behind `requireAdmin`. Unauthenticated requests to `/api/admin/stats`, `/users`, `/files`, `/audit`, `/services`, and `/consent/export` all returned 401 Unauthorized with a generic message and no data leakage. The middleware distinguishes authentication (401) from authorization (403 for non-admin principals).

P-03 Robust anti-automation on authentication

Login and registration validate a reCAPTCHA token *before* any credential processing, returning "Missing reCAPTCHA token." to scripted requests. Because the same generic response precedes any user-existence check, **username enumeration is not possible**, and credential stuffing is blocked at the gate. A failed-attempt rate limiter (20 per 15-minute window, not counting successes) provides further brute-force resistance.

P-04 Algorithm-pinned, constant-time JWT verification

The token verifier recomputes the HMAC-SHA256 signature over the header and payload using a server-only secret and compares it in constant time, **ignoring the token's declared algorithm**. This makes `alg:none` downgrade and HS/RS confusion attacks infeasible, and the constant-time comparison thwarts signature timing oracles. Expiry is enforced.

P-05 Comprehensive transport & browser security headers

- **HSTS** with `max-age ≥ 2 years`, `includeSubDomains`, and `preload` on the frontend; present on the API.
- **Strong CSP** (`object-src 'none'`, `base-uri 'self'`, `frame-ancestors 'none'`, `upgrade-insecure-requests`).
- `X-Frame-Options: DENY` and `frame-ancestors 'none'` — clickjacking defense.
- `X-Content-Type-Options: nosniff`; `modern X-XSS-Protection: 0`; `Referrer-Policy` and `Permissions-Policy` set.

P-06 No information disclosure

Probes for `.env`, `.git/config`, `package.json`, path-traversal variants, and JavaScript source maps (`.js.map`) returned 403/404. Error responses are terse JSON (`{"error": "Not found"}`) with no stack

traces, framework banners, or internal paths. The `x-powered-by` header is disabled.

P-07 Defensive file-upload handling (AI tools)

The AI endpoints require authentication, apply an upload rate limiter, cap uploads via multer (`fileSize` and `files: 1`), use in-memory storage (no temp-file path exposure), validate text inputs with zod length bounds, and restrict images to an explicit MIME allowlist (returning 415 for unsupported types).

P-08 Privacy-preserving data retention

Processed files carry a TTL — one hour for guests, 24 hours for registered users — enforced both lazily on access and by a periodic sweeper that purges expired records from the index, storage, and database. This minimises the window of exposure for any stored output.

P-09 Unguessable resource identifiers

Output files are addressed by `nanoid()` identifiers (~126 bits of entropy), and the storage key is additionally sanitized to a safe character set. Enumeration of other users' files by guessing identifiers is computationally infeasible.

P-10 Observability & audit trail

Requests are tagged with a propagated `x-request-id`, administrative mutations are written to an audit log with actor IP and user-agent, and consent records are maintained — supporting incident response and compliance.

11 Test Cases Executed (Coverage Matrix)

The following matrix records the principal test cases executed during the engagement and their outcomes. "Pass" indicates the control behaved securely; "Finding" indicates a recorded weakness.

#	Category	Test case	Technique	Result
1	CORS	Arbitrary origin reflection with credentials	Spoofed Origin header	Pass
2	CORS	Allowlisted origin handling	Legitimate origin	Pass
3	CORS	Disallowed preflight error handling	OPTIONS w/ bad origin	Finding F-03
4	AuthZ	Admin stats without session	Unauthenticated GET	Pass
5	AuthZ	Admin users list without session	Unauthenticated GET	Pass
6	AuthZ	Admin audit/consent export without session	Unauthenticated GET	Pass
7	AuthZ	Destructive admin delete (withheld)	Not executed (safety)	N/A
8	AuthZ	File download object-level authorization	Source + behaviour	Finding F-01
9	AuthN	Username enumeration via login	Existing vs. fake email	Pass
10	AuthN	Credential stuffing / brute force	reCAPTCHA + rate limit	Pass
11	AuthN	reCAPTCHA enforcement order	Token-less login	Pass
12	AuthN	Auth endpoint rate limiting	Inspect RateLimit headers	Pass
13	Session	JWT alg:none / algorithm confusion	Source review	Pass
14	Session	JWT signature comparison timing	Source review	Pass
15	Session	JWT expiry & claim hardening	Source review	Finding F-07
16	Session	Cookie flags (HttpOnly/Secure/SameSite)	Header inspection	Pass
17	Injection	Content-Disposition header injection	Source + crafted name	Finding F-02
18	Injection	CRLF / response splitting	Crafted filename	Pass
19	Injection	Input validation (text tools)	zod schema review	Pass
20	Injection	SQL/NoSQL injection surface	Prisma parameterization	Pass
21	Info-disc.	Environment file exposure	GET /.env and variants	Pass
22	Info-disc.	VCS metadata exposure	GET /.git/config	Pass
23	Info-disc.	Source map exposure	GET *.js.map	Pass
24	Info-disc.	Stack trace / framework banner	Error responses	Pass

25	Info-disc.	Path traversal on static fetch	Encoded traversal	Pass
26	Config	Security header suite (frontend)	Header inspection	Pass
27	Config	Security header suite (API)	Header inspection	Pass
28	Config	HSTS & preload	Header inspection	Pass
29	Config	Clickjacking defenses	XFO + frame-ancestors	Pass
30	Config	CSP strength (script-src)	Policy review	Finding F-05
31	Design	Rate-limit key trust source	Source + trust proxy	Finding F-04
32	Design	Origin direct reachability	Direct request to origin	Finding F-06
33	Upload	Upload size/count limits	multer config review	Pass
34	Upload	MIME allowlist enforcement	Type validation review	Pass
35	SSRF	AI ingestion outbound control	Source review	Pass
36	Privacy	File retention & auto-deletion	Sweeper + TTL review	Pass
37	IDOR	Resource identifier entropy	nanoid analysis	Pass
38	Method	HTTP method handling on routes	OPTIONS/verb probing	Pass
39	Transport	TLS enforcement & redirect	Scheme inspection	Pass
40	Hardening	x-powered-by suppression	Header inspection	Pass

12 Remediation & Verification Summary

12.1 Changes Implemented This Engagement

Finding	File	Change	Verification
F-01	routes/files.ts	Enforce ownerUserId === req.userId for user-owned files; 404 to non-owners; guests remain capability-based.	Build OK; route returns 404 for unknown id; logic in request path.
F-02	routes/files.ts	Add contentDisposition() helper — ASCII-sanitized filename + RFC 5987 filename*.	Exercised against quote/CRLF/non-ASCII inputs — injection-free.
F-03	middleware/security.ts	CORS reject via callback(null, false) instead of throwing.	Rejected preflight now 200 w/o ACAO; allowlist intact.
F-04	middleware/security.ts	Custom keyGenerator preferring non-forgable client IP on all limiters.	Build OK; limiters wired with key generator.

12.2 Build & Runtime Verification

```
# TypeScript build
$ npm run build
→ prisma generate OK ; tsc -p tsconfig.json → no errors

# Local runtime (guest mode) – control checks
GET /health → {"status":"ok", ... }
OPTIONS /api/auth/login (evil origin) → 200, no ACAO (was 500)
GET /api/tools (evil origin) → 200, no ACAO
GET /api/tools (localhost origin) → ACAO reflected
GET /api/files/doesnotexist123/download → 404
```

Deployment dependency. The remediations for F-01 through F-04 are present in the server codebase and verified locally. They take effect in production only after the next deploy to Render. Until then, live production behaviour reflects the prior build (for example, the disallowed-origin preflight still returns 500 in production).

12.3 Recommended Verification After Deploy

1. Re-run the disallowed-origin preflight against production — expect 200 with no ACAO.
2. Authenticate as User A, generate an output file, capture its id, then attempt download as User B — expect 404.
3. Download a file with a non-ASCII / quote-containing name — expect a well-formed Content-Disposition with both filename and filename*.
4. Confirm authentication throttling still keys correctly for legitimate browser traffic post key-generator change.

13 Strategic Recommendations & Roadmap

13.1 Prioritized Action Plan

Priority	Action	Addresses	Effort	Owner
P1 — Now	Deploy remediated server build to production	F-01..F-04	Low	Owner
P1 — Now	Confirm RECAPTCHA_ENABLED=true remains set in production	P-03	Trivial	Owner
P2 — Near	Restrict direct ingress to Render origin (edge-only)	F-06, F-04	Medium	Owner
P2 — Near	Reject JWTs lacking exp; add iss/aud	F-07	Low	Owner
P3 — Plan	Migrate to nonce-based CSP; remove 'unsafe-inline'	F-05	Medium	Owner
P3 — Plan	Add CI dependency & secret scanning (npm audit, gitleaks)	A06/A08	Low	Owner
P4 — Ongoing	Recurring assessment cadence as tool catalogue grows	Program	Ongoing	Owner

13.2 Defense-in-Depth Enhancements

- **Secret hygiene:** Per prior internal notes, ensure the reCAPTCHA secret that was historically exposed in a chat has been rotated in the reCAPTCHA admin console; never store secrets in source or client code.
- **Server-side anti-malware:** Consider a virus/content-scan hook for uploaded files processed server-side, with results reflected in the existing scanned flag.
- **Security regression tests:** Encode the controls verified here (CORS allowlist, admin 401, reCAPTCHA gate, ownership check, header sanitization) as automated tests so future changes cannot silently regress them.
- **Monitoring & alerting:** Alert on spikes in 401/403, CORS rejections, and rate-limit triggers to detect probing early.
- **Dependency currency:** Track and patch Next.js, React, Express, multer, and Prisma promptly; subscribe to advisories.

13.3 Closing Statement

Arivu's Scrab Tools presents a security posture that is conscientious and well-engineered. The absence of any Critical, High, or Medium findings — combined with correctly implemented, independently-verified core controls — indicates that security has been treated as a first-class concern in the platform's design. The recorded findings are predominantly defense-in-depth refinements, the majority of which were remediated during this engagement. Completing the prioritized actions above, and establishing a lightweight recurring assessment cadence, will keep the platform's risk profile low as it continues to grow.

14 Threat Model (STRIDE)

To contextualise the findings, the platform was modelled against the **STRIDE** framework (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege). The model considers the principal data flows, trust boundaries, and the assets an adversary would target.

14.1 Assets & Adversaries

Asset	Sensitivity	Primary threat actor
User-uploaded documents & processed outputs	High (may contain PII)	External attacker; curious user
Authentication credentials & sessions	High	Credential-stuffing bot; phisher
Administrative functions & user database	Critical	External attacker; privilege escalator
Consent & audit records	Medium (compliance)	Insider; data-scraper
AI tool access (cost-bearing)	Medium (abuse/cost)	Resource-abuse bot
Service availability	Medium	DoS actor (out of scope)

14.2 STRIDE Analysis

Category	Threat scenario	Existing control	Residual / finding
Spoofing	Forging another user's session	Algorithm-pinned, signed JWT; constant-time verify; httpOnly cookie	Low — F-07 advisory
Spoofing	Impersonating the trusted edge to the origin	trust proxy tuning	F-06 / F-04 (direct origin reach)
Tampering	Altering token claims (role escalation)	HMAC signature over header+payload	None — infeasible
Tampering	Manipulating response headers via filename	(now) sanitized Content-Disposition	F-02 remediated
Repudiation	Denying an administrative action	Audit log w/ actor IP + UA; request-id	None significant
Info disclosure	Reading another user's file	(now) ownership check; nanoid entropy; TTL	F-01 remediated
Info disclosure	Leaking config/source/stack traces	Terse errors; no source maps; x-powered-by off	None — verified
Info disclosure	Cross-origin credentialed read	Strict CORS allowlist	None — verified
Denial of service	Auth flooding / brute force	reCAPTCHA + rate limiting	F-04 (key trust) remediated

Denial of service	Oversized uploads	multier size + count caps; in-memory bound	None significant
Elevation	Reaching admin functions as a user	requireAdmin RBAC	None — verified 401/403
Elevation	AI cost abuse by anonymous user	requireAuth + upload limiter + service flag	None significant

14.3 Data-Flow Trust Boundaries

```
[Browser] --TLS→ (B1) [Vercel edge] --TLS→ (B2) [Cloudflare] → [Render origin]
                                     |
(B3) identity boundary: anonymous guest vs authenticated user (JWT)
(B4) authorization boundary: USER role vs ADMIN role (requireAdmin)
(B5) storage boundary: in-memory index + storage driver (+ optional Postgres)
```

Key risks concentrate at B2 (edge→origin trust; see F-06/F-04) and B3/B4 (identity & authorization; verified strong, with F-01 hardening the file path).

The model confirms the engagement's emphasis: identity and authorization boundaries (B3/B4) are well-defended, while the edge-to-origin boundary (B2) is the most valuable hardening opportunity.

15 Privacy & Compliance Considerations

Because the platform processes user documents that may contain personal data and maintains explicit consent records, a brief data-protection lens (aligned to GDPR principles) is included. This is advisory context, not a legal assessment.

15.1 Data-Protection Principle Mapping

Principle	Platform behaviour	Observation
Data minimisation	Optional login; client-side processing for many tools; short retention	Strong — minimal server-side data footprint.
Storage limitation	TTL: guest 1h, user 24h; sweeper purges expired data	Strong — automated deletion enforced.
Integrity & confidentiality	TLS everywhere; HSTS; access controls; (now) file ownership check	Good — strengthened by F-01 fix.
Lawfulness / consent	Consent records maintained & exportable (admin)	Good — auditable consent trail.
Accountability	Audit log with actor, IP, UA, request-id	Good — supports DSAR/incident response.
Purpose limitation	AI inputs sent to model provider for the requested tool only	Review — document third-party processing in a privacy notice.

15.2 Recommendations

- Maintain a clear privacy notice describing third-party processors (e.g., the AI model provider, reCAPTCHA) and retention windows.
- Ensure the data-subject deletion path also evicts the in-memory file index entry, not only durable storage.
- Consider regional data-residency expectations given Singapore (API) and Mumbai/bom1 (frontend) hosting.

16 OWASP WSTG Coverage Checklist

Coverage of the OWASP Web Security Testing Guide control families relevant to this application. Legend: Tested performed this engagement; Review assessed via source/config; N/A not applicable to the architecture.

WSTG ref	Control	Coverage	Result
INFO-02	Fingerprint web server / framework	Tested	Banners suppressed; edge identified.
INFO-05	Review webpage content for info leakage	Tested	No source maps / secrets exposed.
INFO-08	Fingerprint web application framework	Tested	Next.js/Express identified; no version leak.
CONF-01	Network/infrastructure configuration	Review	Edge + origin; F-06 advisory.
CONF-02	Application platform configuration	Tested	Headers strong; errors terse.
CONF-06	HTTP methods	Tested	Method handling correct.
CONF-07	HTTP Strict Transport Security	Tested	HSTS + preload present.
CONF-08	RIA cross-domain policy	Tested	cross-domain-policies: none.
IDNT-04	Account enumeration	Tested	Not possible (reCAPTCHA-first).
IDNT-05	Weak/unenforced registration	Tested	reCAPTCHA-gated.
ATHN-01	Credentials over encrypted channel	Tested	TLS enforced.
ATHN-03	Default/guessable credentials	Review	Admin bootstrapped via env secrets.
ATHN-04	Weak lock-out / brute force	Tested	Rate limited 20/15m; reCAPTCHA.
ATHN-07	Weak password policy	Review	Recommend explicit strength policy.
ATHZ-01	Directory traversal / file include	Tested	Traversal probes 404; key sanitized.
ATHZ-02	Bypassing authorization schema	Tested	Admin routes 401/403.
ATHZ-03	Privilege escalation	Tested	Role from signed JWT; no escalation.
ATHZ-04	Insecure direct object references	Tested	F-01 (hardened); nanoid entropy.
SESS-01	Session management schema	Review	Stateless JWT + refresh cookie.
SESS-02	Cookie attributes	Tested	HttpOnly/Secure/SameSite correct.
SESS-05	Cross-Site Request Forgery	Review	SameSite + CORS + token header.
SESS-09	Session timeout	Review	JWT exp enforced (15m default).
SESS-10	Session puzzling / JWT	Tested	alg-pinned; F-07 advisory.

INPV-01	Reflected XSS	Review	React encoding; strict CSP (F-05).
INPV-02	Stored XSS	Review	No untrusted HTML rendering observed.
INPV-05	SQL injection	Review	Prisma parameterized queries.
INPV-12	Command injection	Review	No shelling out to OS observed.
INPV-15	HTTP splitting/smuggling	Tested	CRLF blocked; F-02 hardened.
INPV-19	Server-Side Request Forgery	Review	No user-controlled outbound URLs.
ERRH-01	Improper error handling	Tested	F-03 (preflight 500) remediated.
ERRH-02	Stack traces	Tested	None exposed.
CRYP-01	Weak transport layer security	Tested	TLS + HSTS preload.
CRYP-04	Weak encryption / signing	Review	HMAC-SHA256; strong secret.
BUSL-09	Upload of malicious files	Review	Size/MIME limits; recommend AV scan.
BUSL-01	Business logic data validation	Review	zod schemas on inputs.
CLNT-13	Cross-origin resource sharing	Tested	Allowlist correct (P-01).
CLNT-09	Clickjacking	Tested	DENY + frame-ancestors none.

17 Detailed Reproduction & Proof-of-Concept

Step-by-step reproduction for each finding, suitable for an engineer to confirm the issue and validate the fix. Commands are illustrative; sensitive or destructive steps are intentionally described rather than executed.

17.1 SCRAB-F-01 — File download object-level authorization

1. Authenticate as **User A**; perform a server-side tool that yields a downloadable output; note the returned file id (an unguessable nanoid).
2. From a separate session authenticated as **User B** (or anonymously), request GET /api/files/<A's id>/download.
3. **Pre-fix**: the bytes are returned (200) regardless of ownership. **Post-fix**: a non-owner of a user-owned file receives 404.

Post-fix expectation

```
GET /api/files/<user-A-file-id>/download (as User B) → 404 "File not found or expired."
GET /api/files/<guest-file-id>/download (any caller) → 200 (capability model retained)
```

17.2 SCRAB-F-02 — Content-Disposition filename injection

1. Process/store a file whose original name contains a double-quote, e.g. in voice";filename="x.exe.
2. Download it and inspect the Content-Disposition response header.
3. **Pre-fix**: the stray quote terminates the value early, allowing a spoofed filename. **Post-fix**: the quote is replaced in the ASCII fallback and the true name is carried safely in filename*.

Post-fix header

```
Content-Disposition: attachment; filename="in voice_;filename=_x.exe";
filename*=UTF-8'"in%20voice%22%3Bfilename%3D%22x.exe"
```

17.3 SCRAB-F-03 — Rejected CORS preflight error handling

1. Send a preflight to a sensitive endpoint with a disallowed origin.
2. **Pre-fix**: server responds 500. **Post-fix**: 200 with *no* Access-Control-Allow-Origin header.

```
curl -i -X OPTIONS -H "Origin: https://evil.example" \
-H "Access-Control-Request-Method: POST" \
https://tools-live.onrender.com/api/auth/login
# Pre-fix: HTTP/1.1 500 Internal Server Error
# Post-fix: HTTP/1.1 200 (no access-control-allow-origin)
```

17.4 SCRAB-F-04 — Rate-limit key from forwarded headers

1. Reach the origin directly (see F-06) and issue repeated failed logins while varying a prepended X-Forwarded-For value.
2. **Pre-fix**: each forged IP receives a fresh quota, diluting the throttle. **Post-fix**: the key generator prefers the non-forgeable platform header for legitimate traffic; the most robust control is origin lock-down (F-

06).

Note: This scenario was reasoned from source and configuration; it was *not* executed as a live brute-force, in keeping with the non-destructive rules of engagement.

17.5 SCRAB-F-05 — CSP unsafe-inline

1. Fetch the frontend and inspect the Content-Security-Policy response header.
2. Observe `script-src 'self' 'unsafe-inline' ...`. Confirm the (accepted) trade-off and plan a nonce-based migration.

17.6 SCRAB-F-06 / F-07 — Origin reach & JWT hardening

1. Resolve `tools-live.onrender.com` directly and confirm it answers without traversing the intended edge path.
2. For F-07, review `verifyJwt()`; confirm it ignores the token `alg` header and checks `exp`; add explicit rejection of tokens lacking `exp`.

18 Attack Scenario Walkthroughs

The following narratives illustrate how an adversary might chain conditions, and why the platform's controls interrupt each chain. They contextualise the residual risk for non-technical stakeholders.

18.1 Scenario — “Steal another user’s document”

Goal: read a registered user's processed file. **Path:** the attacker would need a valid file id. Because ids are 126-bit nanoids, they cannot be guessed or enumerated; the only avenue is out-of-band leakage of a specific id. Even then, post-remediation the download path enforces ownership for user-owned files, returning 404 to a non-owner. **Outcome:** chain broken at two independent points (entropy + ownership). *Residual: minimal.*

18.2 Scenario — “Take over an account by brute force”

Goal: guess credentials. **Path:** automated login attempts are stopped before credential logic by reCAPTCHA, and failed attempts are rate-limited per client. Username enumeration is impossible because the same generic reCAPTCHA error precedes any account lookup. **Outcome:** credential stuffing and enumeration are both defeated. *Residual: minimal.*

18.3 Scenario — “Reach admin functions without rights”

Goal: read the user database or toggle services. **Path:** every admin route requires an authenticated ADMIN principal; the role is carried in a signed JWT that cannot be forged or downgraded. Unauthenticated and standard-user requests receive 401/403. **Outcome:** no horizontal or vertical escalation path observed. *Residual: minimal.*

18.4 Scenario — “Hijack a session via stolen token”

Goal: replay or forge a session. **Path:** tokens are HMAC-signed with a server-only secret and verified in constant time with a pinned algorithm; cookies are httpOnly (resisting XSS theft) and the CORS allowlist prevents cross-site credentialed reads. **Outcome:** forging is infeasible; theft requires a separate, unobserved client compromise. *Residual: low.*

18.5 Scenario — “Abuse AI tools to run up cost”

Goal: consume paid AI capacity anonymously. **Path:** AI endpoints require authentication, are rate-limited, enforce upload size/MIME limits, and can be disabled via an admin service flag. **Outcome:** abuse is bounded and attributable. *Residual: low.*

Appendix A HTTP Evidence Transcripts

Representative, redacted request/response excerpts captured during testing. Volatile values (request IDs, cookies, rays) truncated.

A.1 API root — header surface

```
HTTP/1.1 404 Not Found
access-control-allow-credentials: true
cross-origin-opener-policy: same-origin
referrer-policy: no-referrer
strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
x-permitted-cross-domain-policies: none
x-xss-protection: 0
Server: cloudflare
Set-Cookie: cf_guest=*****; HttpOnly; SameSite=Lax
```

A.2 Frontend — Content Security Policy

```
HTTP/1.1 200 OK
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
X-Frame-Options: DENY
Content-Security-Policy: default-src 'self';
  script-src 'self' 'unsafe-inline' https://www.google.com https://www.gstatic.com;
  style-src 'self' 'unsafe-inline'; img-src 'self' blob: data: ... ;
  connect-src 'self' blob: https://tools-live.onrender.com ... ;
  object-src 'none'; base-uri 'self'; form-action 'self';
  frame-ancestors 'none'; upgrade-insecure-requests
```

A.3 CORS — origin reflection test

```
# Disallowed origin — no ACAO granted
GET /api/auth/me   Origin: https://evil-attacker.example
→ (no access-control-allow-origin header)

# Allowlisted origin — reflected
GET /api/auth/me   Origin: https://tools.arivanandhan.in
→ access-control-allow-origin: https://tools.arivanandhan.in
→ access-control-allow-credentials: true
```

A.4 Administrative authorization

```
GET /api/admin/stats      → 401 {"error":"Authentication required."}
GET /api/admin/users      → 401
GET /api/admin/files      → 401
GET /api/admin/audit      → 401
GET /api/admin/services   → 401
GET /api/admin/consent/export → 401
```

A.5 Authentication gate & rate limiting

```
POST /api/auth/login {"email":" ... ","password":"x"}  
→ 400 {"error":"Missing reCAPTCHA token."}  
    ratelimit-limit: 20  
    ratelimit-policy: 20;w=900  
    ratelimit-remaining: 19
```

A.6 Information-disclosure probes

```
GET /.env → 404  
GET /.git/config → 404  
GET /server/.env → 404  
GET /package.json → 404  
GET /_next/static/chunks/main-app.js.map → 403
```

A.7 Remediation retest (local)

```
OPTIONS /api/auth/login Origin: https://evil.example → 200 (no ACA0) # was 500  
GET /api/tools Origin: https://evil.example → 200 (no ACA0)  
GET /api/tools Origin: http://localhost:3000 → ACA0 reflected  
GET /api/files/doesnotexist123/download → 404
```


Appendix B Security Header Analysis

Header	Frontend	API	Assessment
Strict-Transport-Security	2y; includeSubDomains; preload	1y; includeSubDomains	Strong
Content-Security-Policy	Strict (see F-05)	n/a (JSON API)	Strong*
X-Frame-Options	DENY	SAMEORIGIN	Strong
X-Content-Type-Options	nosniff	nosniff	Strong
Referrer-Policy	strict-origin-when-cross-origin	no-referrer	Strong
Permissions-Policy	camera(self), mic(), geo()	n/a	Good
X-XSS-Protection	n/a	0 (modern best practice)	Correct
Cross-Origin-Opener-Policy	—	same-origin	Good
x-powered-by	suppressed	suppressed	Good
frame-ancestors	'none'	n/a	Strong

* CSP rated strong overall; the 'unsafe-inline' in script-src is the single weakening element, tracked as F-05.

Appendix C CWE / OWASP Mapping

Finding	CWE	OWASP Top 10 (2021)	OWASP API Top 10 (2023)	WSTG
F-01	CWE-639, CWE-284	A01 Broken Access Control	API1 BOLA	WSTG-ATHZ-04
F-02	CWE-116, CWE-93	A03 Injection	API8 Misconfig	WSTG-INPV-15
F-03	CWE-388	A05 Misconfiguration	API8 Misconfig	WSTG-ERRH-01
F-04	CWE-348, CWE-770	A04 Insecure Design	API4 Resource Consumption	WSTG-ATHN-?
F-05	CWE-1021, CWE-79	A05 Misconfiguration	—	WSTG-CLNT-13
F-06	CWE-693	A05 Misconfiguration	API8 Misconfig	WSTG-CONF-01
F-07	CWE-345	A07 Auth Failures	API2 Broken Auth	WSTG-SESS-10

Appendix D Glossary & References

D.1 Glossary

Term	Definition
BOLA / IDOR	Broken Object Level Authorization / Insecure Direct Object Reference — accessing another principal's resource by referencing its identifier without an ownership check.
CORS	Cross-Origin Resource Sharing — browser mechanism governing cross-origin reads of HTTP responses.
CSP	Content Security Policy — browser policy constraining the sources of executable and embedded content.
CVSS	Common Vulnerability Scoring System — standardized severity scoring (v3.1 used here).
CWE	Common Weakness Enumeration — community catalogue of software weakness types.
HSTS	HTTP Strict Transport Security — forces HTTPS and protects against downgrade.
JWT	JSON Web Token — signed token format used for stateless authentication.
RBAC	Role-Based Access Control — authorization based on assigned roles (e.g., USER vs. ADMIN).
SSRF	Server-Side Request Forgery — coercing the server into making attacker-controlled outbound requests.
Capability URL	An access model where possession of an unguessable URL/identifier is the authorization.

D.2 References

- OWASP Web Security Testing Guide (WSTG) v4.2
- OWASP Top 10 (2021) & OWASP API Security Top 10 (2023)
- Penetration Testing Execution Standard (PTES)
- CVSS v3.1 Specification — FIRST.org
- MITRE Common Weakness Enumeration (CWE)
- RFC 6265 (HTTP Cookies), RFC 6797 (HSTS), RFC 6454 (Web Origin), RFC 5987 (Header Field Encoding), RFC 7519 (JWT)

D.3 Disclaimer

A penetration test is a point-in-time, best-effort assessment and cannot guarantee the absence of all vulnerabilities. Findings are based on the information and access available during the engagement window. Remediation should be validated independently, and security should be maintained as an ongoing program rather than a one-time activity.

Appendix E CVSS v3.1 Scoring Rationale

Per-finding base-metric breakdown supporting the assigned severities.

SCRAB-F-01 — BOLA on file download 3.1 LOW

AV	AC	PR	UI	S	C	I	A
Network	High	Low	None	Unchanged	Low	None	None

High attack complexity reflects the need to obtain a 126-bit id out of band; confidentiality impact limited to a single leaked file within its TTL.

SCRAB-F-02 — Content-Disposition injection 2.6 LOW

AV	AC	PR	UI	S	C	I	A
Network	High	Low	Required	Unchanged	None	Low	None

Requires a crafted filename and victim interaction; bounded to download-name spoofing (no splitting).

SCRAB-F-04 — Rate-limit key spoofing 3.7 LOW

AV	AC	PR	UI	S	C	I	A
Network	High	None	None	Unchanged	None	Low	Low

Requires direct origin access and header crafting; partially mitigated by reCAPTCHA on auth.

SCRAB-F-05 — CSP unsafe-inline 3.1 LOW

Scored as a mitigation-strength reduction; not independently exploitable. Severity reflects reduced defense-in-depth should another injection defect arise.

SCRAB-F-03 / F-06 / F-07 — 0.0 INFORMATIONAL

No direct confidentiality, integrity, or availability impact; classified as robustness/architecture/best-practice hardening.

Appendix F Tooling & Methodology Notes

Activity	Approach
HTTP probing	Crafted requests with custom headers (Origin, X-Forwarded-For), method fuzzing, low volume.
Header/TLS analysis	Response-header inspection across frontend and API; scheme/HSTS validation.
Source review	Manual TypeScript review of routers, middleware, JWT, and file services (grey-box).
Local verification	Standalone API runtime (guest mode) to confirm remediations safely.
Build validation	<code>npm run build</code> (Prisma generate + tsc) — zero errors.
Exclusions	No automated exploitation frameworks, no DoS/load tooling, no destructive payloads against production.

F.1 Representative Commands

```
# Header surface
curl -s -I https://tools.arivanandhan.in/
curl -s -I https://tools-live.onrender.com/

# CORS reflection
curl -s -I -H "Origin: https://evil-attacker.example" \
  https://tools-live.onrender.com/api/auth/me

# Admin authorization
curl -s -o /dev/null -w "%{http_code}" \
  https://tools-live.onrender.com/api/admin/users

# Info-disclosure
for p in .env .git/config package.json; do
  curl -s -o /dev/null -w "$p %{http_code}\n" \
    https://tools-live.onrender.com/$p
done
```

Appendix G Remediation Code Diff

The exact source changes implemented and verified during this engagement, provided as evidence and for review. Two files were modified; 42 insertions, 2 deletions.

G.1 server/src/routes/files.ts

```
@@ download handler @@
    const { meta, buffer } = result;
+
+ // Access control: files owned by a registered user are scoped to that user -
+ // knowing the (unguessable) id is not sufficient. Guest outputs
+ // (ownerUserId === null) stay capability-based. Return 404 (not 403) so a
+ // non-owner cannot confirm the id exists.
+ if (meta.ownerUserId && meta.ownerUserId !== req.userId) {
+   throw new HttpError(404, "File not found or expired.");
+ }
+
    res.setHeader("Content-Type", meta.mimeType);
-   res.setHeader("Content-Disposition", `attachment; filename="${meta.filename}"`);
+   res.setHeader("Content-Disposition", contentDisposition(meta.filename));
    res.setHeader("Content-Length", String(buffer.length));

+ function contentDisposition(name: string): string {
+   const fallback = name.replace(/[\x20-\x7e]/g, "_").replace(/["\\]/g, "_") ||
"download";
+   const encoded = encodeURIComponent(name);
+   return `attachment; filename="${fallback}"; filename*=UTF-8'${encoded}`;
+ }
```

G.2 server/src/middleware/security.ts

```
@@ imports @@
+ import { clientIp } from "../lib/client-ip";
+
+ function rateLimitKey(req: Request): string {
+   return clientIp(req) ?? req.ip ?? "unknown";
+ }

@@ cors origin callback @@
    logger.warn({ origin }, "Blocked by CORS");
-   return callback(new Error("Not allowed by CORS"));
+   // Reject by withholding CORS headers rather than throwing (avoids a 500).
+   return callback(null, false);

@@ rate limiters (apiRateLimiter, uploadRateLimiter, authRateLimiter) @@
    legacyHeaders: false,
+   keyGenerator: rateLimitKey,
    message: { ... },
```

Build status: npm run build — Prisma generate + TypeScript compile completed with zero errors. Changes

verified in a local runtime as documented in §12.

End of report. This document is CONFIDENTIAL. The findings reflect the state of the assessed systems and source on 25 June 2026 and should be revalidated after remediation deployment and any significant subsequent change. Report ID: SCRAB-PT-2026-06-25-01 · Version 1.0 (Final).